Software Architecture and Design Specification — Online Examination Portal

Course: UE24CS341A — Software Engineering, PES University, Dept. of CSE
Project: Online Examination System with Multiple-Choice Questions and Evaluation
Team: Kshitij G Shettigar (Backend & Data) · Kartik Basavaraj Meti (Frontend & UX) · Komal S Sajja (QA & Docs)
Version: 3.1 · **Date:** 2026-11-12 · **Status:** Describes the system as built on `main`

Revision history

Version	Date	Author	Change summary
1.0	2026-10-15	Kshitij	Layered architecture, components and interfaces, technology stack, deployment, known limits
2.0	2026-11-03	Kshitij	Adds traceability, security architecture and the design section: sequence diagrams, API design and error handling. Removes the never-built cron command (Deviations D-13)
3.0	2026-11-11	Kshitij	Arranged to the course SAD template: goals and constraints, stakeholders and concerns, risks, STRIDE threat model, UX design, open issues and appendices added. The architecture itself is unchanged
3.1	2026-11-12	Kshitij	Load-test results in §3 risks and §4.6 open issues (Deviations D-18). The architecture is unchanged

Approvals

Role	Name	Signature/Date
Course Coordinator		
Team — Backend & Data	Kshitij G Shettigar	
Team — Frontend & UX	Kartik Basavaraj Meti	
Team — QA & Docs	Komal S Sajja	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Examination Portal. It covers the pattern chosen and why, the components and their interfaces, the security architecture, and the request-level design: sequence diagrams, API, error handling and UX.

1.2 Scope

It covers the 30 functional and 14 non-functional requirements of `docs/SRS.pdf`, as built on `main` at the end of Sprint 2 (build 2d74453) plus the Defect 1 fix of 11 Nov.

The services covered are accounts and roles, courses and enrollment, the question bank, examination set-up and scheduling, the attempt, automatic evaluation, results, and integrity and audit.

1.3 Audience

Developers, QA engineers, security reviewers, the course instructor and evaluators, and whoever maintains the portal after the course.

1.4 Definitions

Term	Meaning
Layer	A group of components that may call only the layer below it: Presentation → Business → Data
Component	One Django app with one responsibility, e.g. `attempts`
Interface	A named set of operations one component provides and another requires, e.g. `IAttempt`
ORM	Django's object-relational mapper, the only path to the database
WAL	SQLite's write-ahead-log mode, which lets readers continue during a write
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
CSRF	Cross-site request forgery
FR / NFR / UC	Requirement and use case IDs of `docs/SRS.pdf`

2. Document Overview

2.1 How to use this document

§3 is the architecture: what the parts are, why this shape, and how it is secured. §4 is the design: how a request moves through the parts, what each endpoint accepts and returns, how errors are handled and how the screens work.

It follows IEEE 1016 (Software Design Descriptions): an architectural viewpoint (§3) followed by interaction, interface and UX viewpoints (§4). Requirement IDs are those of the SRS throughout.

2.2 Related Documents

Document	Where
Software Requirements Specification (FR-001 … FR-030, NFR-001 … NFR-014), with the RTM in §8	`docs/SRS.pdf`
Use case diagram and flows (UC-01 … UC-13)	`docs/Use_Case_Diagram.pdf`, `docs/Use_Case_Flow.pdf`
Software Test Plan (119 cases)	`docs/Test_Plan.pdf`
System Validation Report	`docs/System_Validation_Report.pdf`
Deviations register (every change from these documents)	`docs/Deviations.pdf`

3. Architecture

3.1 Goals & Constraints

Goals.

- **Trustworthy results:** no answer key, timer or score can be read or changed from the browser (SO-1, SO-2).
- **No lost work:** every answer saved within 5 s; service back within 60 s of a crash, with every attempt resumable (FR-019, NFR-004).
- **Responsive at exam scale:** 95% of requests within 2 s at 200 concurrent attempts (NFR-001); a 100-question attempt scored within 5 s (NFR-002).
- **Usable by every student:** WCAG 2.1 AA, any current browser, 360 px screens (NFR-005, NFR-009).

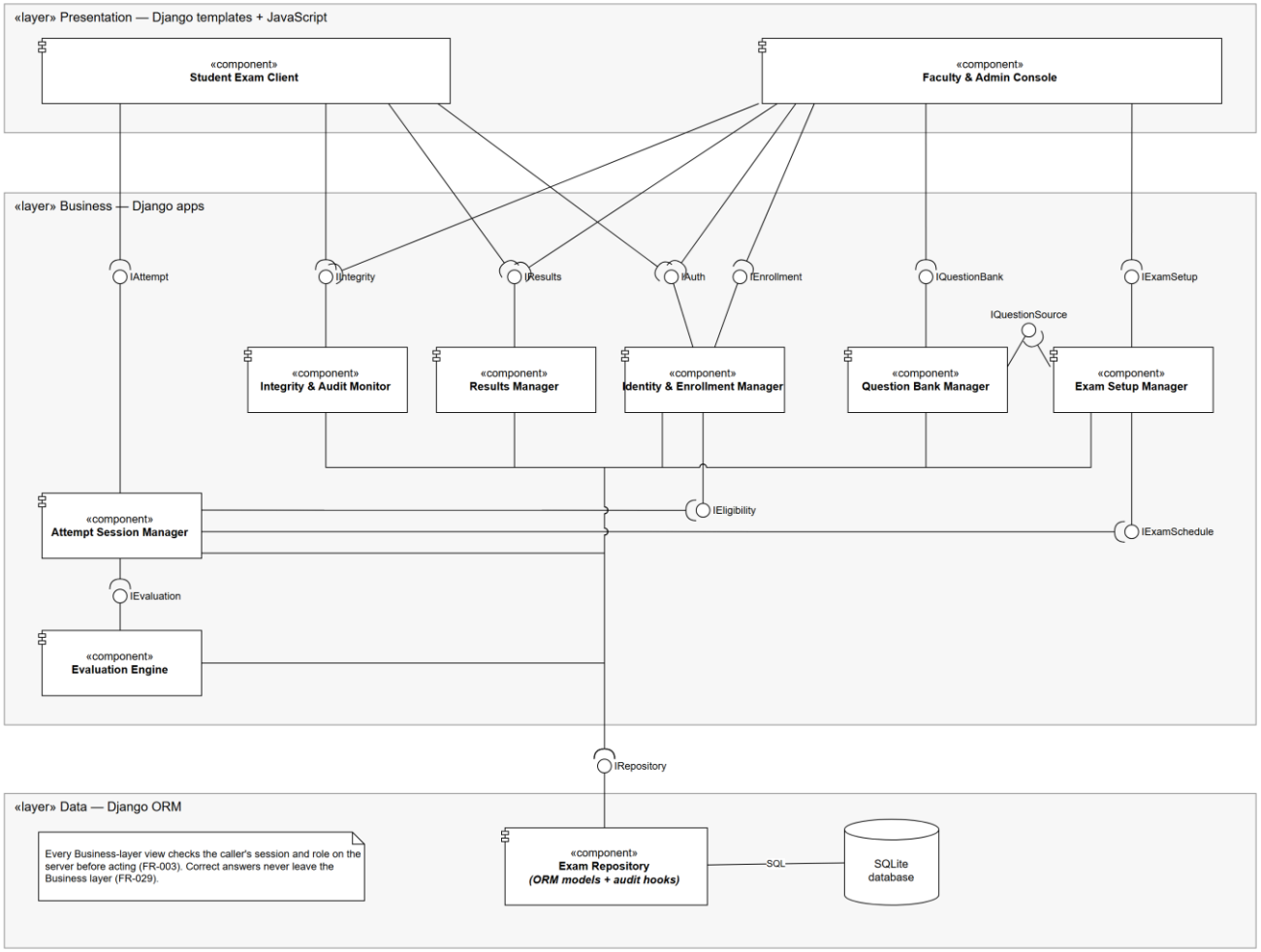
Constraints.

- A team of three and two one-week sprints.
- One Linux virtual machine; no budget for managed services.
- A public repository, so no secret may be committed (NFR-014).
- Every timing rule must read the server clock (FR-017).

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed by
Students	A fair timer; no lost answers; results they can check	Server-side timer (§3.9), autosave and resume (§4.2), per-question scorecard
Faculty	Papers kept private until published; exact scoring; evidence of misconduct	Draft state, Evaluation Engine, focus-loss records
Administrators	Who changed what, and when	Append-only audit trail in the Data layer
Course instructor and evaluators	Requirements traceable to code and tests	§3.8 traceability, `portal/traceability.py`
Developers (the team)	Each member can build a component without breaking another	One Django app per component, interfaces in §3.3

3.3 Component (UML) Diagram



Notation: a ball is a **provided** interface on the component that implements it; a socket is a **required** interface on the component that calls it; a ball in a socket is an assembly connector.

Interface	Provided by	Required by	Kind
`IAuth`	Identity & Enrollment Mgr	Student Exam Client, Faculty & Admin Console	HTTP view
`IEnrollment`	Identity & Enrollment Mgr	Faculty & Admin Console	HTTP view
`IQuestionBank`	Question Bank Mgr	Faculty & Admin Console	HTTP view + JSON endpoint
`IExamSetup`	Exam Setup Mgr	Faculty & Admin Console	HTTP view
`IAttempt`	Attempt Session Mgr	Student Exam Client	HTTP view + JSON endpoints (autosave, timer)
`IIntegrity`	Integrity & Audit Monitor	Student Exam Client (report events), Faculty & Admin Console (view)	HTTP view + JSON endpoint
`IResults`	Results Mgr	Student Exam Client, Faculty &	HTTP view + JSON endpoint

Interface	Provided by	Required by	Kind
		Admin Console	
`IEligibility`	Identity & Enrollment Mgr	Attempt Session Mgr	In-process call
`IExamSchedule`	Exam Setup Mgr	Attempt Session Mgr	In-process call
`IQuestionSource`	Question Bank Mgr	Exam Setup Mgr	In-process call
`IEvaluation`	Evaluation Engine	Attempt Session Mgr	In-process call, same transaction as submission
`IRepository`	Exam Repository	all seven Business components	Django ORM

Count: 7 Presentation→Business, 4 Business→Business, 1 Business→Data. Nothing in the Presentation layer requires `IRepository`. §4.3 lists the HTTP endpoints behind each HTTP interface.

3.4 Component Descriptions

Layer	Component	Django app	Responsibility	Requirements
Presentation	**Student Exam Client**	templates + `attempts/static`	Dashboard, exam screen, server-driven countdown, question palette, autosave and focus-loss listeners, scorecard	FR-014, FR-017, FR-018, FR-019, FR-027, FR-028
Presentation	**Faculty & Admin Console**	templates	Course and enrollment screens, question editor, exam setup and scheduling, results publishing, integrity and audit views	FR-004, FR-005, FR-007–FR-013, FR-026
Business	**Identity & Enrollment Manager**	`identity`	Registration, sign-in, roles, account lockout, course enrollment, eligibility checks	FR-001–FR-006, FR-015, NFR-011, NFR-012
Business	**Question Bank Manager**	`questionbank`	Questions, options, tags, versioning of questions in use	FR-007, FR-008, FR-009
Business	**Exam Setup Manager**	`examsetup`	Examination details and policy, question selection, Draft/Published state, schedule window	FR-010–FR-013, NFR-010
Business	**Attempt Session Manager**	`attempts`	Start attempt, server timer, answer-free paper payload, autosave, resume, submit and auto-submit	FR-015–FR-021, FR-029
Business	**Evaluation Engine**	`evaluation`	Scores a submitted attempt: positive and negative marks, total, percentage, pass/fail	FR-022, FR-023, FR-024
Business	**Results Manager**	`results`	Withholds results until published; publish and unpublish; scorecards	FR-025, FR-026, FR-027
Business	**Integrity & Audit Monitor**	`integrity`	Records focus-loss events; shows events and the audit trail to Faculty and Administrators	FR-028, FR-030
Data	**Exam Repository**	`repository`	Django ORM models over the database; writes the append-only audit entry on every change	FR-030, NFR-013

3.5 Chosen Architecture Pattern and Rationale

We chose a Layered architecture (Presentation / Business / Data).

Style	Fit for this system	Verdict
Layered	One deployable on one server; every rule (timer, eligibility, scoring) sits in one Business layer over one consistent database	**Chosen**
Microservices	Independent scaling is not needed at 200 concurrent attempts (NFR-001), and splitting attempts and scoring across services breaks the single transaction that submission needs (FR-020, FR-022)	Rejected — operational cost with no requirement that pays for it
Client-Server	Topologically true — browsers are the clients of one server — but	Absorbed — the layered

Style	Fit for this system	Verdict
	says nothing about how the server is organised	server *is* the server

Reasons for the choice.

1. **One consistent record for each attempt.** Starting, saving, submitting and scoring all read and write the same attempt row (FR-016, FR-019, FR-020, FR-022). In one Business layer over one database that is a single transaction; across services it would need distributed coordination.
2. **The rules live on the server, behind one layer.** The timer comes from the server clock (FR-017), eligibility from server time and enrollment (FR-015), and the answer key never leaves the server (FR-029). The browser cannot reach or alter any of them.

Security advantage. The Presentation layer has **no path to the database**. It can only call Business-layer interfaces, and every Business-layer view checks the caller's session and role first (FR-003).

The Attempt Session Manager builds the Student's question payload without the correct option. The Presentation layer has no other route to it, so the key cannot leak through the page (FR-029, verified by `IT_035`).

Performance benefit. Evaluation runs inside the server, on data already in the same database, as soon as the attempt is submitted — no network hop between services.

Scoring a 100-question attempt is one query for the responses and one bulk update of their marks, well within 5 s (NFR-002). Every page reads through one repository with indexed foreign keys, which is what NFR-001 is load-tested against.

3.6 Technology Stack & Data Stores

Layer	Technology	Why
Presentation	Django templates, HTML/CSS, plain JavaScript (timer display, autosave on every change, `visibilitychange` and `blur` listeners)	Server-rendered pages keep the answer key server-side by default; no separate front-end build
Business	Django 5 (Python 3.12) — one Django app per Business component	Built-in authentication, password hashing (NFR-011), CSRF protection and ORM (NFR-013)
Data	Django ORM over SQLite in WAL mode	Relational data — enrollment, attempts and scores — in one file database; the ORM keeps a later move to PostgreSQL a settings change
Automation	GitHub Actions — runs `python manage.py test` on every push and pull request	Course requirement (Sprint 1); a failing test blocks the merge

Data stores. One SQLite database holds every entity: users, courses, enrollments, questions and their versions, examinations, attempts and responses, scores, focus-loss events and audit entries.

Auto-submission without a background worker. An expired attempt is submitted and evaluated the first time any request touches it. Every attempt view, and the autosave, timer and focus-loss endpoints, call `close_if_expired()` first.

That call stamps the submission at the exact expiry time (FR-020, FR-022). The open attempt page also reloads itself at zero, so a Student still on the page sees the submission at once. §3.7 records the limit this leaves.

Correction in 2.0: version 1.0 also described a `close_expired_attempts` command run by cron. It was never built, because the request-time check covers every attempt that is looked at again (Deviations D-13).

Deployment. One Linux virtual machine runs the Django application under Gunicorn behind Nginx (HTTPS), with the SQLite file on the same machine and its clock synchronised by NTP.

Secrets such as SECRET_KEY come from environment variables, never from the repository (NFR-014). After a restart the service is back within 60 seconds, and every attempt resumes from its stored start time and saved responses (NFR-004).

3.7 Risks & Mitigations

Risk	Mitigation	Upgrade path
Single server — a single point of failure	The requirement is recovery (NFR-004), not zero downtime; autosave and server-held start times make every attempt resumable	Two app servers behind the load balancer once the database moves to PostgreSQL
SQLite allows one writer at a time	The 200-user load test measured about 50 answer saves per second, each a short write, plus a write on a Student's first view of each question; WAL mode with `IMMEDIATE` transactions and a 20 s busy timeout queues them. Three of four runs met NFR-001; one stalled on the lock at p95 3.6 s with 2 timeouts (Deviations D-18)	Switch the DATABASES setting to PostgreSQL; no code changes
An expired attempt whose Student never returns is submitted only when a request next touches it	The Student can never act on it, since every Student request calls `close_if_expired()` first; it is stamped at its true expiry. Until then Faculty views list it as not submitted	A cron-run sweep calling `close_if_expired()` on every open attempt past its expiry
A browser event misread as misconduct	Focus-loss records are evidence for Faculty, not an automatic penalty; Defect 1 (navigation counted as focus loss) fixed on 11 Nov	—

3.8 Traceability to Requirements

Each component, the requirements it realises and the Test Plan cases that verify them. Every FR maps to exactly one owning Business component. `portal/traceability.py` holds the case-to-FR matrix in code, and a test fails if any case loses its automated test (NFR-007). The full RTM is §8 of the SRS.

Component	Requirements	Test Plan cases
Identity & Enrollment Manager	FR-001–FR-006, NFR-011, NFR-012	UT_001–UT_021, IT_001–IT_011, IT_038, ST_001–ST_004
Question Bank Manager	FR-007, FR-008, FR-009	UT_022–UT_027, IT_012–IT_014, ST_005
Exam Setup Manager	FR-010–FR-013, NFR-010	UT_028–UT_035, IT_015–IT_017, ST_006, ST_007
Attempt Session Manager	FR-014–FR-021, FR-029	UT_036–UT_052, IT_018–IT_028, IT_035, ST_007–ST_010
Evaluation Engine	FR-022, FR-023, FR-024, NFR-002	UT_053–UT_058, IT_028–IT_031, ST_011
Results Manager	FR-025, FR-026, FR-027	UT_059–UT_063, IT_032–IT_034, ST_012
Integrity & Audit Monitor	FR-028, FR-030	UT_064–UT_068, IT_036, IT_037, ST_013
Exam Repository	FR-030, NFR-004, NFR-013	IT_036, IT_037; `portal/test_restart.py`, `portal/test_security.py`
Student Exam Client	FR-017, FR-018, FR-028, NFR-005, NFR-009	UT_042–UT_044, IT_021, IT_022, UT_064–UT_067; `portal/test_responsive.py`, `portal/test_accessibility.py`

3.9 Security Architecture

Trust boundaries. The **browser** is untrusted: everything it sends is validated on the server, and nothing it reports (its clock, which option is correct) is believed.

The **database** is reached only through the Django ORM in the Data layer. Business components never build SQL.

Threat model (STRIDE).

STRIDE threat	Example in this system	Mitigation	Requirement
Spoofing	Signing in as another user by guessing the password; reusing a fixated session	Salted PBKDF2 hashes; lockout after 5 failures; session id rotated at sign-in	NFR-011, NFR-012, FR-002
Tampering	Changing the PC clock to gain time; forging an autosave or publish request from another	Timer from the server clock only; CSRF token on every POST; saves into a submitted attempt refused	FR-017, FR-020, NFR-013

STRIDE threat	Example in this system	Mitigation	Requirement
	site		
Repudiation	Faculty denying they edited a question after the exam	Append-only audit entry with actor, time and old and new values; no screen can edit one	FR-030
Information disclosure	Answer key in the page source; another Student's result by URL; a stack trace on error	Answer-free payload; owner filter on every attempt and result query; generic error pages	FR-029, FR-006, FR-025, NFR-013
Denial of service	Password guessing at volume; autosave bursts locking the database	Account lockout; WAL mode with a busy timeout so writes queue instead of failing	NFR-012, NFR-001
Elevation of privilege	A Student calling a Faculty endpoint directly	Default-deny `RoleMiddleware`: a view without a role declaration is refused to everyone	FR-003

Controls, layer by layer.

Threat	Control	Where it lives	Requirement
Using another role's screens or endpoints	`RoleMiddleware` denies by default: a view without a `@roles(...)` declaration is refused to everyone, so a forgotten decorator fails closed	`identity/access.py`, every request	FR-003
Reading another Student's attempt or result	Every attempt and result query filters on `student=request.user`, so another Student's id returns 404 or 403	`attempts/views.py`, `results/views.py`	FR-006, FR-025
Leaking the answer key to the browser	The paper payload selects only option `id` and `text`; `is_correct` never leaves the Business layer	`attempts.views.paper`	FR-029
Extending time with the client clock	Remaining time = server `started_at` + duration − server now; the browser only displays it	`attempts.services.expires_at`	FR-017, FR-020
Password theft from a database leak	Django's salted PBKDF2-SHA256 hashing; passwords are never logged	Django auth, `repository.User`	NFR-011
Password guessing	5 failed sign-ins in 15 minutes lock the account for 15 minutes, checked before the password	`identity/lockout.py`, `identity/forms.py`	NFR-012
Cross-site request forgery	`CsrfViewMiddleware` on every POST; the page scripts post the form's CSRF token with each autosave and focus-loss report	`portal/settings.py`	NFR-013
SQL injection	ORM only: a code search finds no `.raw()`, `cursor()` or `.extra()` outside tests	all apps	NFR-013
Information leak in errors	DEBUG is off unless `DJANGO_DEBUG=1`; custom 400/403/404/500 handlers render one generic page	`portal/error_handlers.py`	NFR-013
Secrets in the public repository	`SECRET_KEY` is read from `DJANGO_SECRET_KEY`; the app refuses to start without it	`portal/settings.py`	NFR-014
Clickjacking	`XFrameOptionsMiddleware` sends `X-Frame-Options: DENY`	`portal/settings.py`	NFR-013
Untraceable changes	Every create, update and delete of a user, question, exam, answer key or result writes an `AuditEntry`; no view edits or deletes one	`repository/audit.py`	FR-030

Where security is validated. §5.1 of the Test Plan maps each control to the test cases and manual checks that verify it. The System Validation Report records the results of the manual checks.

4. Design

4.1 Design Overview

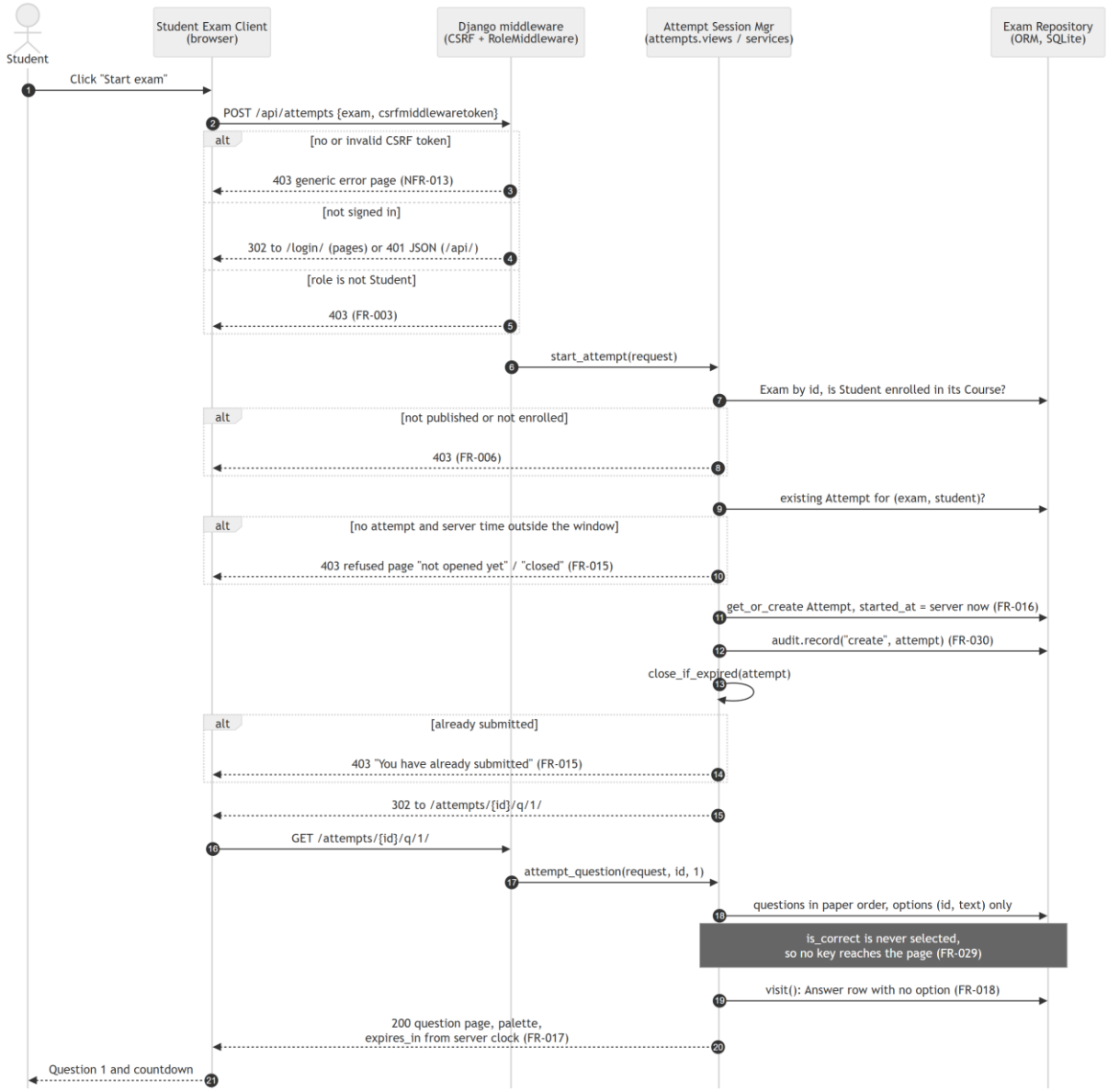
Every request enters through a Presentation-layer view, which checks the role, calls one Business component and renders a page or returns JSON. Business components share data only through the Repository, and call each other only through the four in-process interfaces of §3.3.

State changes that must happen together — submitting and scoring an attempt — run in one database transaction. The design favours server-rendered pages: the only JavaScript is the timer display, autosave and focus-loss reporting on the exam screen.

4.2 UML Sequence Diagrams

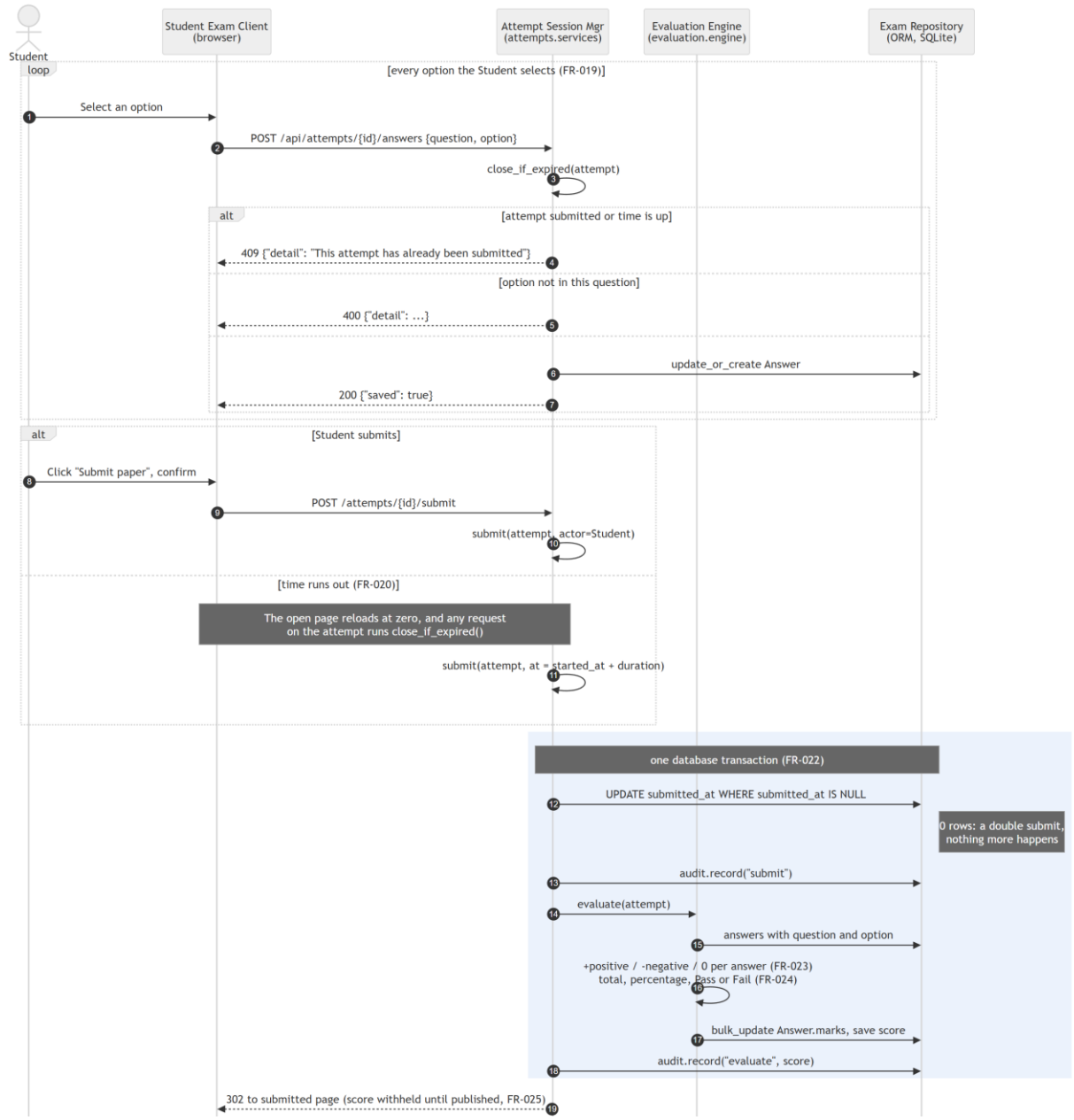
Three interactions carry the system's core rules. Each diagram follows the code on `main`; numbers on the arrows are message order, and `alt` frames show the refusals.

Sequence 1 — Start an attempt and receive the paper (UC-08: FR-003, FR-006, FR-015, FR-016, FR-017, FR-018, FR-029). The attempt row and its server start time exist before any question is sent, and the question payload is built without the answer key.

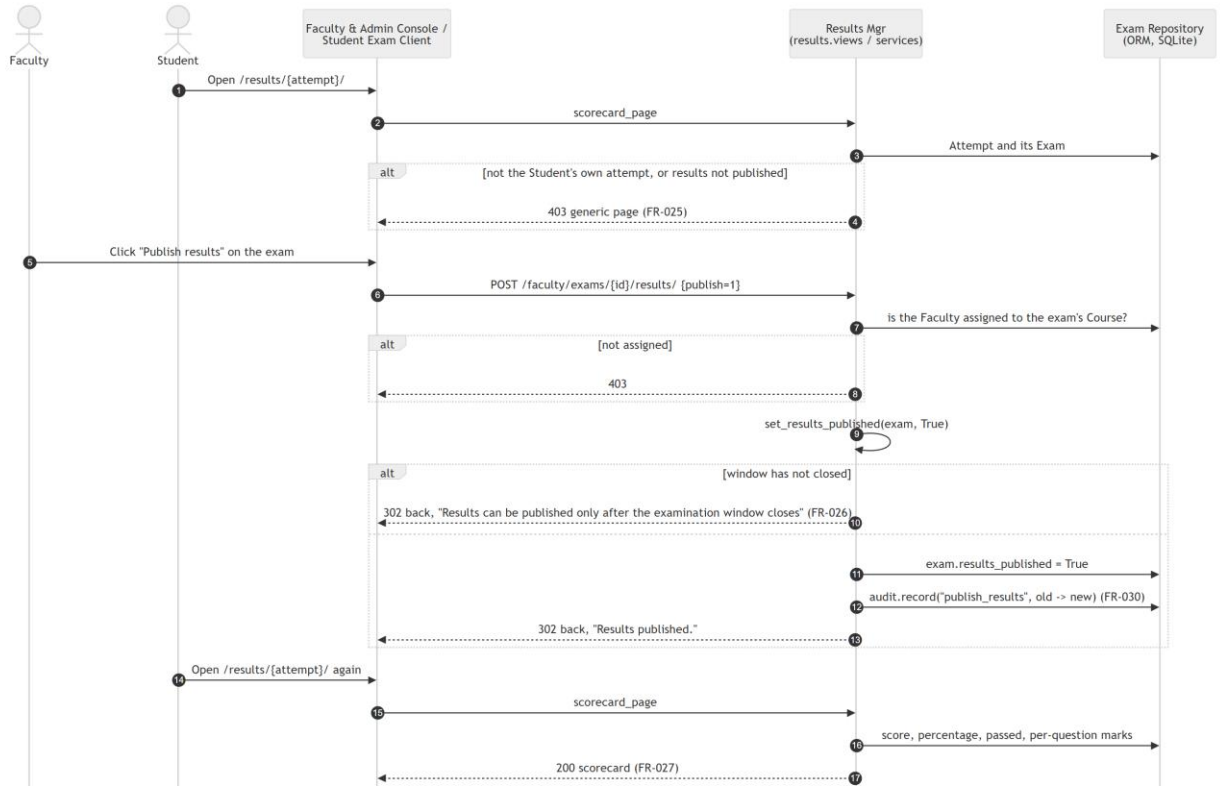


Sequence 2 — Autosave, submission and automatic evaluation (UC-09, UC-10, UC-11: FR-019, FR-020, FR-022, FR-023, FR-024, FR-030). Manual submission and the time-out share one `submit()`.

It marks the attempt submitted and evaluates it in one transaction, so a submitted attempt is never left without a score, and a second submit changes nothing.



Sequence 3 — Publish results and view the scorecard (UC-12: FR-025, FR-026, FR-027, FR-030). A score stays hidden until the course's Faculty publishes the results, which is allowed only after the window has closed.



4.3 API Design

The portal is server-rendered, so most endpoints return HTML pages and form posts redirect (Post/Redirect/Get). The JSON endpoints under `/api/` serve the in-page scripts: autosave, the timer and focus-loss reporting.

Every endpoint needs a signed-in user of the listed role (FR-003), and every POST needs the CSRF token (NFR-013). The tables below define the interfaces of the Attempt Session Manager, Integrity & Audit Monitor, Results Manager and Question Bank Manager, and the pages of every component.

Conventions.

- Resources are addressed by integer id in the path (`/attempts/{id}/`).
- JSON responses carry either the result or `{"detail": "<message>"}`.
- Form posts answer `302` on success and re-render the form with inline errors on invalid input.
- An attempt id that is not the caller's own answers `404`, so the endpoint does not reveal that it exists. An exam, course or result the caller is not allowed to see answers `403`.

JSON endpoints

Method & path	Role	Request	Success	Errors	Requirement
POST `/api/attempts`	Student	form `exam=<id>`	`302` to the paper	`403` not enrolled, outside the window, already submitted · `404` no such exam	FR-015, FR-016
POST `/api/attempts/{id}/answers`	Student	form `question`, `option` (empty clears), optional `marked_for_review` `0/1`	`200 {"saved": true}`	`400` malformed or option not in the question · `409` attempt already submitted or out of time	FR-018, FR-019
GET `/api/attempts/{id}/remaining`	Student	—	`200 {"remaining_seconds": n}` (server clock; an expired attempt is submitted first)	`404` not the caller's attempt	FR-017, FR-020

Method & path	Role	Request	Success	Errors	Requirement
POST `/api/attempts/{id}/focus-lost`	Student	no body	`204`	`409` attempt already submitted · `404` not the caller's attempt	FR-028
GET `/api/results/{attempt}`	Student	—	`200 {"score", "percentage", "passed"}`	`403` another Student's attempt or results not published	FR-025
POST `/api/questions`	Faculty	JSON question with options	`201 {"id": n}`	`400 {"errors": {field: [messages]}}` invalid JSON or fields	FR-007, FR-008

Unauthenticated calls to `/api/` get `401 {"detail": "Authentication required."}` rather than a redirect, because a script cannot follow a sign-in page.

Page endpoints

Area	Paths	Role	Requirement
Accounts	`GET/POST /register/`, `/login/`, `POST /logout/`	public	FR-001, FR-002, NFR-008, NFR-012
Dashboards	`/student/`, `/faculty/`, `/admin/`	own role	FR-002, FR-014
Courses and enrollment	`/admin/courses/` (+ `new/`, `{id}/edit/`, `{id}/archive/`), `/faculty/courses/`, `/courses/{id}/roster/`, `/courses/{id}/enroll/`, `/courses/{id}/unenroll/{student}/`	Admin, Faculty	FR-004, FR-005
Data erasure	`POST /admin/students/{id}/erase/`	Admin	NFR-008
Question bank	`/faculty/questions/` (+ `new/`, `{id}/edit/`, `{id}/delete/`)	Faculty	FR-007–FR-009
Examinations	`/faculty/exams/` (+ `new/`, `{id}/`, `{id}/edit/`, `{id}/questions/`, `{id}/publish/`, `{id}/schedule/`)	Faculty	FR-010–FR-013
Attempt	`/student/exams/`, `/student/exams/{id}/`, `/attempts/{id}/`, `/attempts/{id}/q/{n}/`, `POST /attempts/{id}/submit`	Student	FR-006, FR-014–FR-021
Results	`POST /faculty/exams/{id}/results/` (`publish=1` or `0`), `/results/{attempt}/`	Faculty, Student	FR-025–FR-027
Integrity and audit	`/faculty/exams/{id}/integrity/`, `/admin/audit/?entity=&entity_id=`	Faculty, Admin	FR-028, FR-030

4.4 Error Handling, Logging & Monitoring

Status codes. The same failure gets the same answer everywhere:

Situation	Response	Example
Not signed in	Page: `302` to `/login/?next=…` · API: `401` JSON	Opening `/student/` after sign-out
Signed in, wrong role or someone else's resource	`403` generic page (API: `403`)	A Student opening `/faculty/questions/`
Missing CSRF token	`403`	A forged POST from another site
No such resource	`404` generic page	`/attempts/9999/`
Malformed input	Form: re-rendered with inline field errors · API: `400` with `detail` or `errors`	Question with seven options
Valid request, wrong state	API: `409` with `detail` · Form: refusal page or message	Autosave after the time is up
Business rule refusal at start	`403` refusal page naming the reason	"This examination has not opened yet"
Unexpected exception	`500` generic page; the transaction in progress is rolled back	—

Rules.

3. **Fail closed.** A view that declares no roles is refused; an unknown attempt state is treated as closed; a request to save into a submitted attempt is refused, never applied.
4. **No internals in responses.** DEBUG is off, and the four handlers in `portal/error_handlers.py` render one generic `errors/error.html` page with no stack trace or SQL (NFR-013).
5. **State changes are atomic.** Submission and evaluation run in one transaction, so a crash leaves the attempt either open with its saved answers or submitted with a score, never half-done (FR-022, NFR-004). The conditional `UPDATE … WHERE submitted_at IS NULL` makes a double submit, or a manual submit racing the time-out, a no-op.
6. **Saves are idempotent.** Autosave is an upsert keyed on (attempt, question), so sending the same response twice leaves one row. A `409` makes the page reload, which shows the submitted page. The script does not retry a save that failed on the network; the palette cell only turns Answered once the server confirms the save, so the Student can see what is stored.
7. **Database contention waits instead of failing.** SQLite runs in WAL mode with `IMMEDIATE` transactions and a 20-second busy timeout, so concurrent autosaves queue rather than raise "database is locked" (NFR-001).
8. **Messages say what to do.** A refusal names the reason in the Student's terms (not opened yet, closed, already submitted); form errors sit next to the field that caused them.

Logging. No sensitive data is logged: passwords never appear in a log (NFR-011). The portal has no custom logging configuration; Django's default writes request errors to the server console, which is Gunicorn's log in deployment.

Business events that matter later are recorded in the database rather than in log files: the append-only audit trail (FR-030) and the focus-loss events (FR-028). Faculty and Administrators read them in the portal.

Monitoring. GitHub Actions runs the full suite and a missing-migrations check on every push and pull request. Runtime monitoring of the server is not built; it is listed in §4.6.

4.5 UX Design

- **One question per page** with a palette of every question's state — Not Visited, Not Answered, Answered, Marked for Review — so a Student can pace and review a long paper (FR-018).
- **The countdown is the server's.** The page only displays the remaining time and reloads at zero to show the submitted state (FR-017, FR-020).
- **No save button.** Each choice is saved as it is made; the palette cell turns Answered only once the server confirms, so what is shown is what is stored (FR-019).
- **Role dashboards.** Each role lands on its own dashboard; Students see their papers grouped as Upcoming, Live and Completed (FR-014).
- **Accessible by keyboard and screen reader.** A skip link to the main content on every page, labelled option controls and 4.5:1 text contrast (NFR-009). axe reports 0 AA violations.
- **Small screens.** Every page works at 360 px wide without horizontal scrolling (NFR-005).
- **Plain refusals.** Messages name the reason ("This examination has not opened yet"), and form errors sit next to the field.

4.6 Open Issues & Next Steps

Item	Next step
One of four 200-user load runs stalled on SQLite's write lock (NFR-001)	Re-run the load test before any real deployment; if the stall recurs, move to PostgreSQL (§3 risks)
Open instructor questions: flat or fractional negative marking; inclusive pass mark	Built as flat and inclusive (SRS §2.6); change the Evaluation Engine if the answer differs
No runtime monitoring	Add a health endpoint and uptime alerting on the deployment server
Single server, SQLite	Move to PostgreSQL, then add a second app server (§3.7)
Expired attempts closed only on the next request	Add the cron-run sweep of §3.7 if Faculty views must show them as submitted at once

5. Appendices

5.1 Glossary

Attempt, Examination, Window, Publish and Focus loss are defined in SRS §1.4. Layer, Component, Interface, ORM, WAL, STRIDE and CSRF are defined in §1.4 of this document.

5.2 References

- IEEE 1016-2009, Software Design Descriptions.
- ISO/IEC/IEEE 42010, Architecture description.
- OWASP Top 10 (2021) — the web threats behind NFR-013.
- Django 5.2 documentation — security, authentication and the ORM.
- `docs/SRS.pdf`, `docs/Test_Plan.pdf`, `docs/Deviations.pdf`.

5.3 Tools

Tool	Used for
draw.io (generated from Python)	UML component and use case diagrams
Mermaid	UML sequence diagrams
GitHub, GitHub Projects and GitHub Actions	Repository, backlog and CI
Playwright, axe-core, Locust	Browser validation, accessibility audit, load script